Lietišķo datorsistēmu programmatūra(English)(1),25/26-P Architecting an Adaptive AI Agent using Design Patterns

Opened: Monday, 16 March 2026, 12:00 AM

Due: Monday, 30 March 2026, 12:00 AM

1. Assignment Overview

The goal of this assignment is to design and implement an adaptive Artificial Intelligence (AI) agent using the **Google Gemini API**. Unlike simple procedural scripts, this task requires you to architect the system using established **Software Engineering design patterns** and **SOLID principles**.

You will build a **Personal Assistant agent** that maintains conversation history, reasons about user requests, and autonomously decides when to use external tools (*function calling*) to retrieve information or perform actions. The main focus of this assignment is the **software architecture** of the system, ensuring it is modular, extensible, and maintainable.

2. Theoretical Background

AI Agents and Software Architecture

Modern LLM-based agents typically consist of three main components:

- **Planning** – deciding what actions to perform

- **Memory** – storing conversation context
- **Tool Use** – interacting with external systems

Building a robust AI agent requires moving beyond monolithic or "spaghetti" code and applying well-known software architecture principles.

In this assignment you will apply several **Gang of Four (GoF) Design Patterns** commonly used in modern AI systems:

1. **Strategy Pattern**
   Defines a family of algorithms (tools) and makes them interchangeable. The agent can dynamically choose which tool strategy to execute based on the decision generated by the LLM.
2. **Factory / Registry Pattern**
   Used to instantiate and manage available tools. Instead of hardcoding tool logic inside large `if-else` blocks, a registry maps tool names to their implementations.
3. **Observer Pattern (Optional / Bonus)**
   Can be used to monitor agent state changes (for example logging tool usage, updating a UI, or tracking token usage) without tightly coupling logging logic to the main agent loop.

The Agent Loop (ReAct Pattern)

The execution logic of many AI agents follows the **ReAct (Reason → Act → Observe)** pattern:

1. Receive user input
2. Reason about what needs to be done
3. Act by calling a tool if necessary
4. Observe the tool output
5. Repeat the reasoning process until a final answer is produced

3. Requirements and Specifications

You must implement an interactive **Command Line Interface (CLI)** application that acts as the AI agent.

Architectural Requirements (Strictly Enforced)

1. **Separation of Concerns (SRP)**
   The code must be separated into clear logical components such as:
   - Agent
   - MemoryManager
   - ToolRegistry
   - Tool Interface / BaseTool
2. **Tool Interface (OCP & DIP)**
   You must define an abstract base class or interface for tools. Adding a new tool must not require modifying the core Agent class (Open/Closed Principle).
3. **Tool Registry**
   A registry or factory must dynamically register tools and execute them based on the tool name returned by the LLM.

Functional Requirements

1. **Natural Language Interface**
   The agent must interact with the user using natural language.
2. **Contextual Memory**
   The agent must remember previous conversation turns within the same session.
3. **Tool Integration**
   The agent must support at least **four tools**:
   - You may implement example tools such as Calculator, Weather, or Time.
   - You must create at least **two custom tools** (for example: translation, reading local files, news retrieval, etc.).
4. **Adaptive Execution**
   The agent must autonomously decide whether to answer directly or call an external tool.
5. **Robust Error Handling**
   The architecture must gracefully handle:
   - API errors
   - Invalid tool arguments generated by the LLM
   - Unknown tool requests

4. Implementation Guide

Step 1: Environment Setup

1. Ensure **Python 3.10+** is installed.
2. Install required packages:

   ```
   pip install google-generativeai requests
   ```

3. Create a **Gemini API key** using Google AI Studio.
4. Set the API key as an environment variable:
   - **Linux / macOS**
     `export GEMINI_API_KEY="your_api_key"`
   - **Windows**
     `set GEMINI_API_KEY="your_api_key"`

Step 2: Study the Architectural Template

Review the provided file `agent_architecture_template.py`. The system should be structured into separate components:

- **BaseTool** – abstract class defining the tool interface
- **ToolRegistry** – manages tool registration and execution
- **MemoryManager** – stores conversation history
- **Agent** – orchestrates the Reason → Act → Observe loop

Step 3: Implement Custom Tools

1. Create two classes that inherit from `BaseTool`.
2. Implement the `execute()` method containing the tool logic.

3. Implement the `get_declaration()` method that returns the JSON schema required for Gemini function calling.
4. Register the tools inside `ToolRegistry`.

Step 4: Testing the Architecture

Test the robustness of your architecture:

- Ask questions that require multiple tools.
- Test failure scenarios (for example requesting weather for a non-existent city).
- Verify that the agent continues functioning even when errors occur.

5. Evaluation Criteria

Criterion	Description	Weight
Software Architecture & Patterns	Correct implementation of SOLID principles and design patterns. No monolithic tool logic.	40%
Agent Functionality	The agent communicates with Gemini, maintains memory, and executes the Reason → Act → Observe loop.	30%
Custom Tool Implementation	At least two custom tools are implemented correctly with valid schemas.	20%
Code Quality & Error Handling	Clean structure, documentation, type hints, and reliable error handling.	10%

6. Useful Resources

- Refactoring.Guru – Design Patterns
- Google Gemini API Documentation
- SOLID Principles in Python

Good luck! Focus on designing a system that is easy to extend. If adding a new tool feels difficult, revisit your architecture.

Add submission

Submission status

Submission status	No submissions have been made yet

Grading status	Not graded
Time remaining	1 day 5 hours remaining
Last modified	-
Submission comments	▶ Comments (0)

◄ Software Architecture, Design, and Patterns

Jump to...

AI Agent Usage English ►